

# PowerMax CSI Migration Runbook

Per-node cutover from static iSCSI provisioning to Dell CSI PowerMax dynamic provisioning — Path B: array-side data copy via TimeFinder SnapVX

|                  |                                                                       |
|------------------|-----------------------------------------------------------------------|
| Scope            | One Kubernetes worker node, single application, multiple static PVs   |
| Prerequisite     | CSI Driver for Dell PowerMax already installed cluster-wide (Phase 0) |
| Data copy method | Array-level SnapVX full copy (no host-side data movement)             |

---

## Before the Maintenance Window

### 1. Confirm CSI is already installed cluster-wide

Driver, reverse proxy, StorageClass(es) matching your SRP/SLO, and port groups pre-created on the array (Phase 0 from the migration strategy).

### 2. Inventory the app's volumes

For each static PV currently in use, record: device ID (symdev), current size, mount path inside the pod, and which PVC/StorageClass will replace it. Request the new dynamic PVC at a size  $\geq$  the existing device's capacity — mismatched geometry complicates the array-side copy.

### 3. Set volumeBindingMode to Immediate

For the target StorageClass used in this migration, set `volumeBindingMode: Immediate`, not `WaitForFirstConsumer`. `WaitForFirstConsumer` delays the actual `CreateVolume` call on the array until a pod consumes the PVC — but the device needs to exist and sit idle, unattached, before data is copied into it. Immediate binding creates the array device as soon as the PVC is applied.

### 4. Agree a rollback trigger

Agree with the app owner, before starting, what condition triggers a revert to the static volumes and a retried window — not live debugging in a broken state.

# Runbook

## Step 1 — Freeze and shut down the application

```
kubectrl scale statefulset <app> -n <ns> --replicas=0
# or: kubectrl scale deployment <app> -n <ns> --replicas=0
```

Confirm pods are fully terminated, then confirm the node has released the block devices — a lingering multipath map or mount will block a clean copy:

```
kubectrl get pods -n <ns> -o wide          # confirm gone

ssh <node>
mount | grep <static-mount-path>          # should be empty
multipath -l                             # confirm no active I/O on the static maps
```

## Step 2 — Provision the new dynamic PVCs (device only, no pod yet)

Apply PVC manifests referencing the CSI StorageClass, sized per the inventory from prep. Do not deploy the application against these yet.

```
kubectrl apply -f new-pvcs.yaml
kubectrl get pvc -n <ns> -w              # wait for Bound
```

Because volumeBindingMode is Immediate, Bound here means the device already exists on the array — no pod required.

## Step 3 — Map source and target devices

For each PVC, pull the array device ID out of the PV so the storage team can match it to the source static device:

```
kubectrl get pv <new-pv-name> -o jsonpath='{.spec.csi.volumeHandle}{"\n"}'
# format is roughly csi-<ClusterPrefix>-<volNamePrefix>-<volID>-<arrayID>-<deviceID>
```

Cross-reference <deviceID> in Unisphere against the static source device IDs from the pre-migration inventory. Build a source→target device pair list for every volume the app uses.

## Step 4 — Array-side copy (storage team, SnapVX)

If the app has multiple volumes that must be time-consistent with each other (e.g. data + log), snapshot them together as one operation, not one device at a time — this guarantees a single consistent point in time across the set.

```
# Establish a snapshot across all source devices as one atomic set
symsnapvx -sid <SymmID> -devs <src1,src2,src3> establish -name migrate_<app>_<date>

# Link to the new target devices and start a full background copy
symsnapvx -sid <SymmID> -devs <src1,src2,src3> -lndevs <tgt1,tgt2,tgt3> \
  -snapshot_name migrate_<app>_<date> link -copy

# Monitor copy progress until each pair shows "Copied"
symsnapvx -sid <SymmID> list -linked -detail
```

Wait for 100% / Copied state on every pair before proceeding — don't cut the application over on a partial copy.  
Once confirmed:

```
# Unlink now that the full copy is complete and independent
symsnapvx -sid <SymmID> -devs <src1,src2,src3> -lndevs <tgt1,tgt2,tgt3> \
  -snapshot_name migrate_<app>_<date> unlink

# Optional: remove the snapshot session once confident a re-copy won't be needed
symsnapvx -sid <SymmID> -devs <src1,src2,src3> terminate -snapshot_name migrate_<app>_<date>
```

This step happens entirely on the array — the Kubernetes node and CSI driver aren't involved, so there's no risk of the node plugin trying to format the target mid-copy.

## Step 5 — Node cutover (Host / Initiator Group swap)

This is where the “can't be in two Hosts at once” constraint gets resolved for this node.

```
kubectl cordon <node>
```

Storage team:

- Remove the node's IQN(s) from the legacy static Host/Initiator Group.
- Confirm the static masking view no longer includes this node (leave the volumes and SG intact — don't delete anything yet).

On the node, clean up the now-stale static entries (leave the general multipath.conf settings — CSI needs multipathing too):

```
multipath -f <static-device-mapname> # flush the stale map(s)
# remove any node-specific static-only stanzas from /etc/multipath.conf if present
systemctl reload multipathd

kubectl uncordon <node>
```

No manual Host/IG creation needed here — the CSI driver auto-creates the node's Host on this array the first time a pod with a CSI-backed volume schedules to it, which happens in the next step.

## Step 6 — Deploy the application against the new PVCs

Update the app manifest to reference the new PVC names, then scale back up:

```
kubectl apply -f app-with-new-pvcs.yaml
kubectl scale statefulset <app> -n <ns> --replicas=<N>
```

Watch the node plugin logs during first mount to confirm it detects the existing filesystem via blkid and mounts directly rather than running mkfs — the copied data should already carry a valid filesystem, and reformatting would destroy it. Verify this behavior in the Phase 1 pilot before relying on it in production cutovers.

```
kubectl logs -n powermax -l app=csi-powermax-node -c driver --tail=100
```

## Step 7 — Validate

- App starts cleanly, no fsck/mount errors.
- Data integrity check against a checksum or row-count baseline captured *before* shutdown in Step 1 (agree this baseline with the app owner ahead of the window).
- Confirm PVC persistentVolumeReclaimPolicy: Retain is set on the new PVs — cheap insurance during the soak period.

## Step 8 — Soak period, then decommission

Keep the old static devices mapped-off-but-intact for an agreed burn-in window (24–72h is typical). Only after validation holds:

- Storage team deletes the old static Storage Group/masking view entries and reclaims the devices.
- Remove the node/app from the static-storage runbook and inventory.

## Rollback (if validation fails in Step 7)

1. Scale the app back to 0.
2. Re-add the node's IQN to the legacy static Host, re-map the static masking view.
3. Redeploy the app against the original static PVCs.
4. Investigate the dynamic-path failure before re-attempting the window — don't retry blind.

---

Open items to confirm with the storage team before the first production run: whether SnapVX operations are driven via symsnapvx/Solutions Enabler CLI as shown, or via Unisphere's GUI SnapVX workflow, and whether the source/target device pairing lookup (Step 3) should be scripted via PyU4V against the REST API once past the pilot.